

Smart College Cafeteria Food Ordering System - Project Documentation

1. Project Overview

The **Smart College Cafeteria Food Ordering System** is a full-stack web application designed to streamline the food ordering process. It solves the problem of long queues and manual errors by digitalizing the entire workflow from student ordering to kitchen preparation.

2. In-Depth Technology Stack Analysis

Backend: Java Spring Boot

- **Version:** Spring Boot 3.2.2 (JDK 21)
- **Why we chose it:**
 - **Robustness:** Spring Boot is industry-standard for enterprise applications due to its reliability and dependency injection.
 - **Scalability:** Changes backend logic to be easily extended without breaking existing features.
 - **Security:** Comes with 'Spring Security', a powerful framework for handling authentication and authorization out-of-the-box.
- **What we implemented:**
 - **REST API:** Created endpoints (`/api/orders` , `/api/auth`) to communicate with the frontend.
 - **Business Logic:** Handled complex rules like "Students can place orders" but "Only Admins can add menu items".
 - **Data Validation:** Ensured no invalid data (like negative prices or empty emails) enters the database.

Frontend: React.js

- **Framework:** React (using Vite for fast build tool)
- **Why we chose it:**
 - **Component-Based:** Allows us to reuse UI parts like the `Navbar` , `Footer` , and `FoodCard` across multiple pages.
 - **Single-Page Application (SPA):** Provides a smooth user experience where the page doesn't reload on every click, feeling like a mobile app.
 - **State Management:** React Hooks (`useState` , `useContext`) make it easy to manage live data like the Shopping Cart.
- **What we implemented:**

- **Dynamic UI:** The Menu page automatically updates when Admin adds a new item.
- **Cart System:** Managed adding/removing items and calculating totals instantly in the browser.
- **Role-Based Views:** The UI changes depending on who is logged in (Top Navbar shows "Kitchen" only for Chefs).

Database: MySQL

- **Type:** Relational Database Management System (RDBMS)
- **Why we chose it:**
 - **ACID Compliance:** Crucial for financial transactions (Orders). We cannot afford to lose order data if the server crashes.
 - **Structured Relationships:** Our data naturally relates: A *User has many Orders*, and an *Order has many OrderItems*. Relational databases handle this best.
- **What we implemented:**
 - **Tables:** Designed *users*, *foods*, *orders*, and *order_items* tables.
 - **Foreign Keys:** Linked *orders* to *users* so we always know exactly who placed which order.

Security: JWT (JSON Web Tokens)

- **Why we chose it:**
 - **Stateless:** The server doesn't need to remember "sessions". The token itself proves who the user is. This is faster and cheaper for the server.
 - **Cross-Domain:** Since our Front-End (Vercel) and Back-End (Railway) are on different servers, cookies are hard to manage. JWTs work perfectly across domains.
- **What we implemented:**
 - **Token Generation:** When a user logs in, we generate a secure encrypted string containing their ID and Role.
 - **Authorization Filter:** Every request to a protected route (like *Place Order*) checks this token to prevent hackers from faking orders.

Deployment: Railway & Vercel

- **Why we chose them:**
 - **Continuous Deployment:** They automatically re-deploy the new version whenever we push code to GitHub.
 - **Separation of Concerns:** Hosting Frontend and Backend separately scales better.
- **What we implemented:**
 - **Live Production Environment:** The application is live and accessible via public URLs, not just on `localhost`.

3. Database Schema

The database consists of the following key entities:

- **User:** `id` , `email` (Unique), `password` (Encrypted), `role` (STUDENT/ADMIN).
- **Food:** `id` , `name` , `description` , `price` , `imageUrl` , `available` .
- **Order:** `id` , `user_id` (FK), `totalPrice` , `status` (PENDING/READY), `createdAt` .
- **OrderItem:** `id` , `order_id` (FK), `food_id` (FK), `quantity` .

4. System Flow (The "Story" of an Order)

1. **Registration:** Student registers -> Backend hashes password -> Saves to MySQL.
2. **Login:** Student logs in -> Backend verifies hash -> Returns JWT Token.
3. **Browsing:** Frontend requests `/api/foods` -> Backend queries MySQL -> Returns JSON list of foods.
4. **Ordering:** Student clicks "Checkout" -> Frontend sends JSON Order -> Backend saves Order & OrderItems -> Returns "Success".
5. **Preparation:** Chef views "Kitchen Dashboard" -> Sees new order -> Marks as "Preparing".
6. **Notification:** Backend sends an email to the Student: "Your order is being prepared!".

5. Future Enhancements

- **Payment Gateway:** Integration with Razorpay for real payments.
- **WebSocket:** For "Instant" status updates without refreshing the page.
- **Mobile App:** Building a React Native version for iOS/Android.

theme: gaia class: lead paginate: true backgroundColor: #fff
backgroundImage: url('https://marp.app/assets/hero-
background.svg')

Smart College Cafeteria

Food Ordering System

Presented by: [Your Name] Technology: Java Spring Boot + React

1. The Problem

Why do we need this system?

- **Long Queues:** Students waste break time waiting in line.
- **Manual Errors:** Cash handling and manual token systems are prone to mistakes.
- **Inefficiency:** Kitchen staff cannot predict demand or track orders efficiently.
- **No Feedback:** Difficult to collect ratings on food quality.

2. The Solution

A Full-Stack Digital Platform

- **Digital Ordering:** Order from anywhere on campus.
- **Real-time Tracking:** See when your food is "Preparing" or "Ready".
- **Admin Dashboard:** easy menu management and sales oversight.
- **Kitchen Display:** Chefs see live orders instantly.

3. Technologies Used

Building a Robust Architecture

- **Backend:** Java 21, Spring Boot 3 (Validations, Security, JPA).
- **Frontend:** React.js (Hooks, Context API), Vite, Bootstrap.

- **Database:** MySQL 8.0 (Reliable Relational Data).
- **Security:** JWT (JSON Web Tokens) for Stateless Authentication.
- **Deployment:** Railway (Backend/DB) + Vercel (Frontend).

4. Key Features

User/Student

- Sign Up / Login / Forgot Password.
- Browse Menu with Categories.
- Add to Cart & Checkouts.
- Order History & Live Status.

Admin/Hostel

- Manage Menu (Add/Edit/Delete Items).
- View All Orders.
- User Management.

5. System Flow (Demonstration)

1. **Login:** User logs in securely.
2. **Order:** User selects "Chicken Biryani" -> Adds to Cart -> Places Order.
3. **Kitchen:** Chef sees order on Dashboard -> Marks "Preparing".
4. **Notification:** User sees status update.
5. **Pickup:** Chef marks "Ready" -> User collects food -> Email Sent.

6. Future Scope

- **Payment Gateway:** Integration with Razorpay/Stripe.
- **Mobile App:** React Native version.
- **AI Recommendations:** Suggest food based on past orders.
- **Inventory Management:** Auto-deduct stock ingredients.

Thank You!

Any Questions?

Smart Cafeteria Project - Viva Voce Questions & Answers

1. System Architecture & Spring Boot

Q1: Why did you choose a "Microservices-like" architecture (separate Frontend/Backend) instead of a Monolith (JSP/Thymeleaf)?

Student Answer: "We chose a decoupled architecture to separate concerns. The React frontend handles the UI/UX, while the Spring Boot backend creates a robust REST API. This allows us to scale them independently—for example, we could replace the React web app with a Mobile App later without changing a single line of Backend code."

Deep Dive Explanation:

- **Separation of Concerns:** Frontend devs don't need to know Java, Backend devs don't need to know CSS.
- **Scalability:** If the UI is heavy, we can add more Vercel instances. If the DB is heavy, we scale Railway.
- **JSON Standard:** They communicate via JSON, which is the universal language of the web.

Q2: What is "Dependency Injection" in Spring Boot and where did you use it?

Student Answer: "Dependency Injection (DI) is where Spring manages our objects (Beans) for us. Instead of creating `new UserRepository()` everywhere, we use `@Autowired`. This makes the code loosely coupled and easier to test."

Deep Dive Explanation:

- **Inversion of Control (IoC):** You give control of object creation to the Spring Container.
- **Example in Code:** In `AuthController.java`, we `@Autowired` the `AuthenticationManager` and `UserRepository`. We didn't instantiate them manually.

2. Database & Data Integrity

Q3: How do you handle "Concurrency" in the kitchen? (e.g., Two chefs updating the same order)

Student Answer: "Currently, our database uses the 'Last Commit Wins' strategy. If two chefs update an order at the exact same millisecond, the MySQL database will process them sequentially. Since the

status update is atomic (updating a single field), the database ensures the row is locked exclusively during the write operation."

Deep Dive Explanation:

- **Atomic Transactions:** Databases are ACID compliant. The 'I' stands for Isolation.
- **Pessimistic Locking:** You lock the row so no one else can read/write until you finish.
- **Optimistic Locking:** You check a `@Version` number column. If it changed since you read it, you throw an error.

Q4: Why use a Relational Database (MySQL) instead of MongoDB?

Student Answer: "Our data is highly structured and relational. An Order *must* belong to a User. An OrderItem *must* link to a Food. SQL enforces these relationships (Foreign Keys), ensuring we never have an 'orphan' order with no owner. MongoDB is better for unstructured data, which doesn't fit our case."

3. Security (JWT)

Q5: Explain how the Login flow works securely. Do you store the password?

Student Answer: "No, we never store plain text passwords. When a user registers, we use `BCrypt` to hash the password before saving it to the database. When they login, the server compares the hash. If valid, we issue a **JWT (JSON Web Token)**. The frontend stores this token and sends it in the `Authorization` header for every subsequent request."

Deep Dive Explanation:

- **Hashing vs Encryption:** Encryption can be reversed (decrypted). Hashing is one-way. You cannot get "password123" back from a `BCrypt` hash.
- **Stateless:** The server doesn't keep a "session file". The Token *is* the session. It contains the UserID and Expiry signed digitally.

4. Frontend (React)

Q6: What is the "Virtual DOM" in React and why is it fast?

Student Answer: "The Virtual DOM is a lightweight copy of the real browser DOM. When data changes (like adding an item to the cart), React updates the Virtual DOM first, compares it to the previous version (Diffing), and then only updates the specific elements that changed in the real DOM. This implies faster performance than reloading the whole page."

Q7: How do you manage State (like the Shopping Cart) across different pages?

Student Answer: "We use the React Context API (`CartContext`). It acts like a global store that wraps our application. Any component (Menu, Cart Page, Navbar) can access and update the cart data without passing props down manually through every level."

5. Deployment

Q8: You deployed on Vercel and Railway. How do they talk to each other?

Student Answer: "They communicate over the public internet using HTTP. The React app on Vercel sends requests to the Railway URL (e.g., `https://.../api`). We configured **CORS (Cross-Origin Resource Sharing)** in the Spring Boot backend to explicitly allow requests from the Vercel domain; otherwise, the browser would block the connection for security."

Deep Dive Explanation:

- **CORS:** A security feature in browsers. It prevents a site at `evil.com` from reading data from your bank API. You must explicitly whitelist `vercel.app` in your backend `SecurityConfig`.

6. Advanced Technical & Situation-Based Questions

Q9: What is the difference between `@Controller` and `@RestController`?

Student Answer: " `@Controller` is used for traditional Spring MVC where you return a View (like a JSP or HTML page). `@RestController` is a specialized version for REST APIs; it automatically assumes that every method returns **Data** (JSON/XML) instead of a View. It implies `@ResponseBody` on every method."

Q10: How does your application assume roles? I see `@PreAuthorize("hasRole('ADMIN')")`. How does that work?

Student Answer: "When the User logs in, the JWT token contains a 'claim' called `role`. When the backend receives a request with that token, the `AuthTokenFilter` extracts this role and tells Spring Security Context who the user is. The `@PreAuthorize` annotation then checks this Context before allowing the method to run."

Q11: (Trick Question) "If I have 1000 orders, and I call `getAllOrders()`, will it crash your server?"

Student Answer: "It might cause performance issues if the dataset is huge. In a real production scenario, I would implement **Pagination** (using `Pageable` in Spring Data JPA) to fetch only 20 or 50 records at a time. For this project scope, fetching all is acceptable, but pagination is the correct enterprise solution."

7. The "Soft Skill" Question (The most important one)

Q12: What was the most challenging bug you faced while building this?

Student Answer: (Choose one of these true stories from your development)

- **Option A (CORS):** "Connecting the Vercel Frontend to the Railway Backend was tough. I faced many CORS errors because the browser blocked the request. I had to carefully configure the `CorsConfigurationSource` bean in Spring Security to allow the specific Vercel domain."
- **Option B (State Management):** "Managing the Shopping Cart state was tricky. When I refreshed the page, the cart would disappear. I had to implement `localStorage` in React to persist the cart items so they survive a page reload."